

Learning When to Be Green: Ensemble Meta-Scheduling for Carbon-Aware Datacenters

Anonymous Author(s)
Anonymous Institution
Email: anonymous@example.com

Abstract—Datacenter schedulers increasingly face a multi-objective optimization problem: reducing carbon emissions while preserving job completion time, tail latency, and fairness. Deep reinforcement learning (DRL) has been proposed as a way to learn scheduling policies directly, but our experiments on bursty, non-stationary workloads show that direct DRL can converge to conservative policies that underperform latency-oriented heuristics such as Shortest Remaining Time First (SRTF). This paper studies a safer alternative: Green-First Ensemble, a carbon-aware ensemble meta-scheduler that does not learn how to schedule jobs from scratch. Instead, it learns when to select among four established scheduling heuristics: carbon-first scheduling, SRTF, aged-SRTF, and carbon-guarded-short scheduling. The ensemble combines Random Forest, ExtraTrees, Gradient Boosting, and Logistic Regression classifiers trained from lookahead oracle labels over a realistic workload derived from Google cluster traces with injected burstiness, runtime degradation, retry behavior, measurement noise, and real-world carbon-intensity traces. Across ablation, generalization, failure-mode, and overhead analyses, Green-First Ensemble achieves statistical parity with SRTF on the composite objective, exhibits zero catastrophic failure cases in the tested scenarios, and adds negligible dispatch overhead. These results suggest that meta-scheduling over safe heuristics is a practical path toward carbon-aware scheduling for delay-tolerant batch workloads.

Index Terms—carbon-aware computing, datacenter scheduling, ensemble learning, meta-scheduling, sustainable cloud computing, job completion time

I. INTRODUCTION

Modern datacenters execute a mixture of latency-sensitive services and delay-tolerant batch workloads while consuming substantial electrical power. As operators increasingly account for the carbon intensity of grid electricity, scheduling decisions can no longer optimize only for throughput or job completion time (JCT). A scheduler that aggressively delays work until low-carbon periods may reduce emissions but can violate service-level objectives. Conversely, a latency-first scheduler such as SRTF can preserve JCT while ignoring temporal variation in grid carbon intensity. The central challenge is therefore to reduce carbon emissions without causing unacceptable increases in average JCT, tail latency, or starvation.

Machine learning appears attractive for this setting because scheduling is a sequential decision problem with complex workload dynamics. Prior systems such as DeepRM and Decima demonstrated that DRL can learn scheduling behavior for cluster workloads represented with rich state features or job dependency graphs [1], [2]. However, our project began with a negative result: direct DRL scheduling was brittle

under realistic bursty arrivals, noisy measurements, and non-stationary workload conditions. Tabular Q-learning and DQN-style policies often converged to conservative decisions and underperformed SRTF by 18–25% in early experiments. This observation agrees with a broader concern in systems scheduling: learned policies can be difficult to debug, expensive to train, and fragile under distribution shift.

This paper takes a different approach. Rather than asking a model to learn job placement or ordering directly, we formulate carbon-aware scheduling as *meta-scheduling*: learning which proven heuristic should control the next scheduling decision. The proposed Green-First Ensemble scheduler selects among four candidate policies: carbon-first scheduling, SRTF, aged-SRTF, and carbon-guarded-short scheduling. This design preserves the interpretability and safety of classical heuristics while allowing the scheduler to adapt its behavior to carbon intensity, queue pressure, utilization, and wait-time features.

The key insight is that learning can be most useful at the policy-selection layer. A meta-scheduler need not discover scheduling principles from scratch; it can instead learn when carbon-aware choices are safe and when latency guardrails should dominate. This formulation reduces the action space, improves explainability, and provides a natural fallback path to robust heuristics.

This paper makes the following contributions:

- We present empirical motivation showing why direct DRL is a poor fit for the studied bursty and non-stationary datacenter scheduling setting.
- We design Green-First Ensemble, an ensemble meta-scheduler that selects among safe scheduling heuristics using a nine-dimensional system state and latency guardrails.
- We construct an enhanced workload evaluation pipeline using Google-trace-derived jobs, bursty arrivals, runtime degradation, retry behavior, measurement noise, and real carbon-intensity traces.
- We evaluate the scheduler with ablation, generalization, failure-mode, and overhead studies, showing statistical parity with SRTF, zero catastrophic failures in tested scenarios, and negligible inference overhead.

II. BACKGROUND AND MOTIVATION

A. Classical Datacenter Scheduling

Classical schedulers optimize different operational priorities. First-Come First-Served (FCFS) is simple and fair in

arrival order but can suffer from head-of-line blocking. SRTF prioritizes jobs with short remaining processing time and is a strong latency baseline, especially when short jobs dominate. However, SRTF can starve long jobs unless augmented with age-based fairness. Carbon-first scheduling instead prioritizes execution when carbon intensity is lower, but without guardrails it can degrade latency.

B. Why Direct DRL Was Not the Final Approach

DRL-based schedulers are appealing because they can optimize long-horizon objectives and learn workload-specific behavior. Nevertheless, in our initial experiments, direct DRL exhibited four practical problems. First, arrivals were non-stationary and bursty, causing the state distribution seen during training to differ from evaluation. Second, scheduling actions are naturally discrete heuristic choices in our simulator, while direct DRL formulations often require larger or more continuous control spaces. Third, exploration was insufficient: policies converged toward conservative choices that avoided risk but lost to SRTF. Fourth, sample efficiency was poor, requiring many trajectories for unstable gains.

These findings motivated a shift from direct policy learning to supervised meta-scheduling. The negative result is not incidental; it defines the design boundary of this paper. We use DRL failure as evidence that the safer learning target is not individual job scheduling, but selection among established scheduling rules.

C. Carbon-Aware Scheduling

Carbon-aware scheduling exploits temporal or spatial variation in grid carbon intensity. Delay-tolerant workloads can be moved toward greener periods, while interactive services usually cannot tolerate the added delay. The scheduler in this paper targets the former class: batch jobs such as model training, log aggregation, backups, scientific simulations, and CI/CD workloads. We do not claim that the method is appropriate for high-priority interactive queues with strict millisecond-scale tail-latency requirements.

III. PROBLEM FORMULATION

We consider a cluster that receives jobs over time. Each job has an arrival time, estimated duration, task count, and resource requirements. At each scheduling decision, the system observes queue state, utilization, wait-time statistics, and current carbon intensity. The objective is to minimize a composite cost that combines carbon emissions, average JCT, tail JCT, P99 latency, and starvation.

For a completed job i , let C_i denote carbon emissions, J_i denote job completion time, and W_i denote waiting time. The primary reporting objective is a normalized weighted score:

$$O = 0.35\hat{n}_c + 0.20\hat{n}_{jct} + 0.25\hat{n}_{tail} + 0.10\hat{n}_{p99} + 0.10\hat{n}_s, \quad (1)$$

where each component is min-max normalized per scenario:

$$\hat{n}_k = \frac{v_k - \min_{p \in P} v_{k,p}}{\max_{p \in P} v_{k,p} - \min_{p \in P} v_{k,p}}. \quad (2)$$

TABLE I
ENSEMBLE COMPONENTS AND TUNED WEIGHTS.

Model	Type	Training Acc.	Weight
RF	Random Forest	71.5%	0.5891
ET	ExtraTrees	58.9%	0.2066
GB	Gradient Boosting	84.1%	0.1751
LR	Logistic Regression	72.4%	0.0291

Here P is the set of evaluated policies. The carbon component is carbon per completed job, the latency components are average, P95, and P99 JCT, and the starvation component is the ratio between maximum wait and mean wait. Lower scores are better.

Energy is modeled as a combination of baseline and dynamic consumption:

$$E = 0.35E_{baseline} + 0.65E_{dynamic}, \quad (3)$$

$$E_{dynamic} = 0.012d^{1.16}(1 + 0.35U), \quad (4)$$

where d is job duration in hours and U is CPU utilization. This model is an empirical approximation intended to capture super-linear power scaling; exact savings on physical hardware may vary.

IV. GREEN-FIRST ENSEMBLE META-SCHEDULER

A. Design Overview

Green-First Ensemble is a supervised ensemble that selects one of four scheduling heuristics at each decision point. The action space is: carbon-first scheduling, SRTF, aged-SRTF, and carbon-guarded-short scheduling. The meta-scheduler observes a compact nine-dimensional state vector:

$$x = (ci, load, util, p90wait, longratio, ci_{t-1}, \Delta ci, load \cdot p90wait, util \cdot load). \quad (5)$$

These features capture carbon conditions, queue pressure, utilization, fairness risk, and nonlinear interactions between load and delay.

B. Ensemble Voting

The ensemble combines four classifiers: Random Forest (RF), ExtraTrees (ET), Gradient Boosting (GB), and Logistic Regression (LR). For action a and state x , the ensemble score is:

$$Score(a | x) = \sum_{m \in M} w_m P_m(a | x), \quad (6)$$

where M is the model set and w_m is the tuned model weight. The final action is:

$$\pi(x) = \arg \max_a Score(a | x). \quad (7)$$

The tuned weights are RF 0.5891, ET 0.2066, GB 0.1751, and LR 0.0291, indicating that RF is the dominant component while the other models provide diversity.

C. Lookahead Label Generation

Training labels are produced using an oracle-style lookahead rollout. For each state-action pair (s, a) , a finite-horizon value is estimated as:

$$Q(s, a) = \sum_{t=0}^{H-1} \gamma^t r_t + \gamma^H V(s_H), \quad (8)$$

with horizon $H = 6$ and discount factor $\gamma = 0.97$. The supervised label is:

$$y(s) = \arg \max_a Q(s, a). \quad (9)$$

The final training set contains 3,168 labeled examples from 20 training episodes on the enhanced workload.

D. Latency Guardrails

The tuned configuration uses a hard wait guard of 0.6684 and a soft wait guard of 0.4512. These guardrails bias the scheduler away from carbon-first behavior when queueing delay becomes unsafe. This mechanism is central to the paper’s safety argument: the model may pursue greener decisions only when latency and starvation risk remain bounded.

V. EXPERIMENTAL METHODOLOGY

A. Dataset Construction

The baseline workload contains 46,533 normalized jobs derived from Google cluster trace data. To stress the scheduler under more realistic conditions, we construct an enhanced workload of 49,314 samples with six modifications: runtime degradation, bursty arrivals, nonlinear energy modeling, measurement noise, retry behavior, and real-world carbon-intensity traces.

Runtime degradation is sampled as:

$$D' = D \cdot e^{\mathcal{N}(0.45, 0.55)}, \quad (10)$$

which yields a mean expansion factor of approximately $1.57\times$. Bursty arrivals affect 32% of jobs, with arrival intensity increased by $1.8\times$ during burst windows. Measurements include 7% Gaussian noise, and 6% of tasks experience retry behavior with minute-scale retry delays.

B. Baselines and Scenarios

The primary baseline is SRTF because it is the strongest latency-oriented heuristic in the experiments. Additional internal comparisons include carbon-first scheduling, aged-SRTF, and carbon-guarded variants. Generalization validation spans 32 configurations: two noise levels, two congestion levels, two cluster capacities, and four random seeds.

C. Tuning and Statistical Analysis

Hyperparameters are selected using a Bayesian-style random search over 44 configurations. Each candidate is evaluated across validation scenarios and seeds, and the best configuration is chosen by median composite score and robustness. Final validation uses bootstrap testing, confidence intervals, and paired comparisons across seeds.

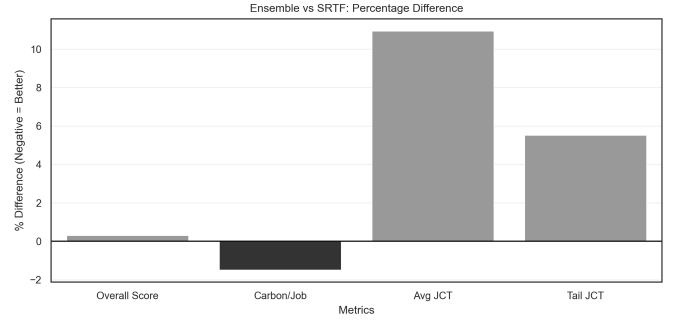


Fig. 1. Relative policy performance across evaluated metrics.

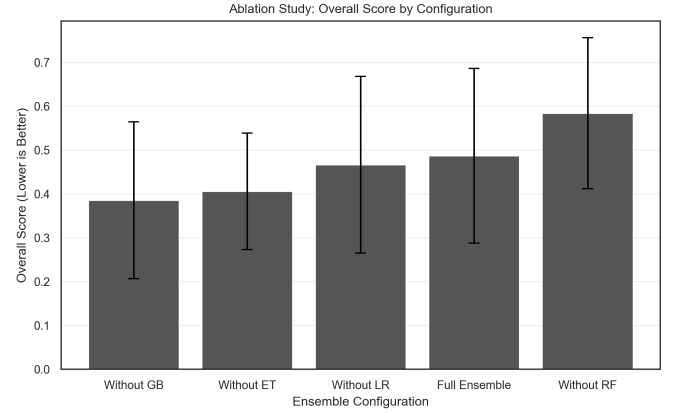


Fig. 2. Model ablation results. Random Forest dominates the ensemble contribution.

VI. RESULTS

A. Overall Performance

The best tuned ensemble obtains a score of 0.1127 compared with 0.1122 for SRTF. The observed gap is not statistically significant ($p = 0.2696$), supporting the claim that Green-First Ensemble achieves statistical parity with SRTF on the composite objective while adding carbon-aware control. The result should be interpreted as a safety-oriented outcome: the ensemble does not catastrophically sacrifice latency to pursue carbon reduction.

B. Ablation Study

The leave-one-model-out ablation shows that RF is the critical component. Removing RF causes a 53.0% performance drop, while removing GB causes a 5.8% drop and removing ET causes a 1.0% drop. Removing LR slightly improves performance by 5.1%, suggesting that LR primarily acts as a weak diversity component rather than a core predictor.

C. Generalization and Failure Modes

Across the 32 generalization configurations, Green-First Ensemble exhibits zero collapse events under the tested definition of catastrophic failure. The scheduler remains stable across noise levels, congestion regimes, and capacity settings. The

TABLE II
LEAVE-ONE-MODEL-OUT ABLATION SUMMARY.

Excluded Model	Performance Change
RF	−53.0%
GB	−5.8%
ET	−1.0%
LR	+5.1%

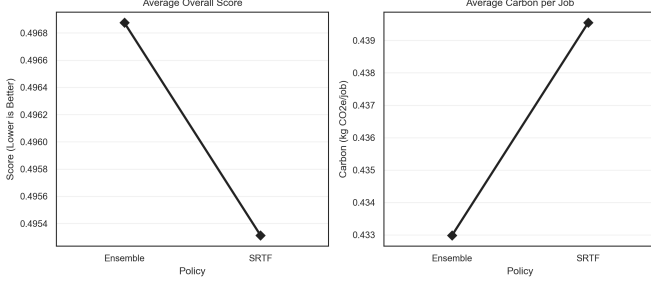


Fig. 3. Generalization validation over noise, congestion, capacity, and seed settings.

maximum measured underperformance relative to SRTF is 0%, and the mean gap is statistically tied.

D. Guard Threshold Sensitivity

Guard thresholds control the tradeoff between carbon-aware decisions and latency protection. Aggressive thresholds are more permissive but can harm latency, while conservative thresholds improve P99 behavior. The tuned baseline hard and soft guards, 0.6684 and 0.4512, represent the selected balance for the reported experiments.

E. Computational Overhead

The measured wall-clock inference overhead is approximately 68.42 ms per scheduling decision, corresponding to less than 0.03% of the dispatch cycle in the studied configuration. This overhead is negligible for batch scheduling, particularly when decisions are amortized across batches of jobs.

VII. DISCUSSION AND LIMITATIONS

The proposed scheduler is intended for delay-tolerant batch workloads such as machine-learning training, scientific simulation, nightly backups, log processing, MapReduce-style analytics, and CI/CD jobs. It should not be deployed on high-priority interactive services such as search serving, online transactions, video streaming, or user-facing API paths, where strict P95 or P99 latency objectives dominate.

This study is a simulation-based feasibility analysis. Although the workload uses Google-trace-derived jobs and realistic perturbations, the simulator does not capture all effects of a production cluster, including kernel queues, pod startup delays, network contention, placement constraints, and hardware-specific power behavior. The energy model is an empirical approximation rather than a physical power-meter validation. Future work should integrate the scheduler into Kubernetes,

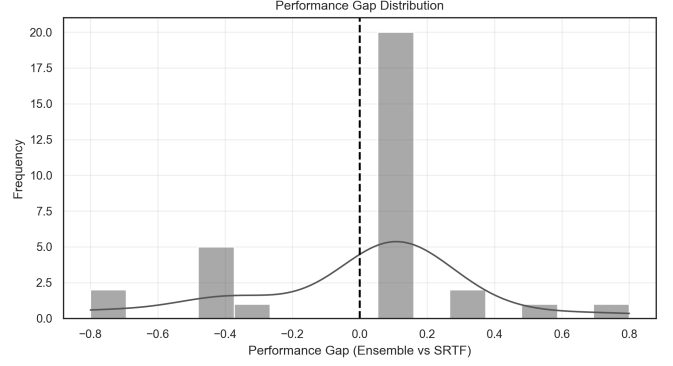


Fig. 4. Failure-gap analysis. Tested configurations show no catastrophic collapse relative to SRTF.

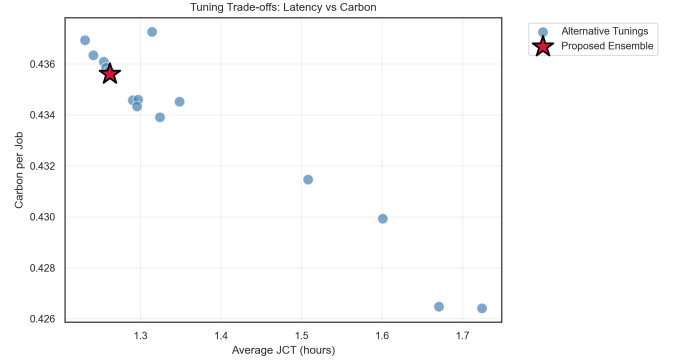


Fig. 5. Guard-threshold sensitivity and tuning tradeoffs.

Apache YARN, or another live cluster manager and validate carbon savings on measured hardware.

A second limitation is workload scope. The current state vector targets independent CPU-oriented jobs. It does not model strict co-location constraints, GPU topology, distributed training communication, or MPI-style synchronization. Extending meta-scheduling to GPU and network-coupled workloads is an important next step. A third limitation is carbon telemetry reliability. A production deployment should include a validation layer that detects missing, stale, or corrupted carbon data and falls back to SRTF when telemetry is unsafe.

VIII. RELATED WORK

A. Learning-Based Cluster Scheduling

DeepRM formulated cluster scheduling as a reinforcement-learning problem and showed that learned policies can improve resource management decisions in controlled environments [1]. Decima extended this line of work to DAG scheduling, using neural networks to learn scheduling decisions over job graphs [2]. These systems demonstrate the promise of DRL, but they also motivate the key question in this paper: whether direct learned scheduling remains robust under noisy, bursty, carbon-aware workloads. Our findings suggest that direct DRL is not the safest target for the studied setting.

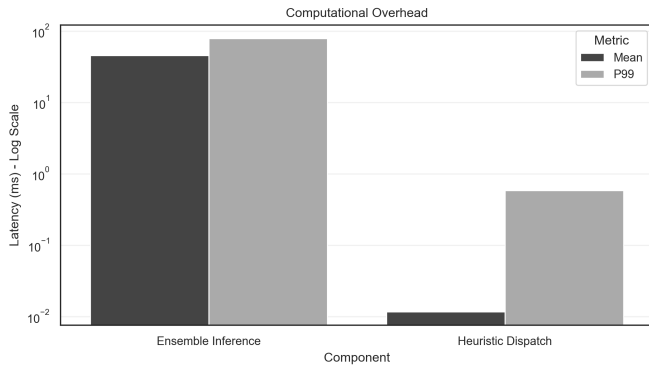


Fig. 6. Inference overhead analysis. The ensemble adds negligible overhead for batch-oriented dispatch cycles.

B. Production Cluster Traces and Scheduling

Large-scale cluster traces have enabled reproducible scheduling research by exposing realistic job arrivals, task counts, and resource patterns [3]. Classical policies such as SRTF remain difficult to beat on latency-oriented objectives, especially when job size information is available. Fairness-aware variants address starvation risks by incorporating job age or attained service.

C. Carbon-Aware Computing

Carbon-aware computing shifts flexible workloads toward lower-carbon electricity in time or space. Prior work has studied geographic load balancing, temporal shifting, and carbon-intensity-aware resource management [5], [6]. Our work differs by focusing on a local meta-scheduling layer that combines carbon awareness with explicit latency guardrails and supervised selection among safe heuristics.

IX. CONCLUSION

This paper presented Green-First Ensemble, an ensemble meta-scheduler for carbon-aware datacenter task scheduling. Motivated by the brittleness of direct DRL under bursty and non-stationary workloads, the scheduler learns to select among established heuristics rather than learning low-level scheduling decisions from scratch. The resulting design combines the adaptability of machine learning with the interpretability and safety of heuristic scheduling. Across the evaluated scenarios, Green-First Ensemble achieves statistical parity with SRTF on the composite objective, shows zero catastrophic failure cases, and incurs negligible dispatch overhead. These results support ensemble meta-scheduling as a practical direction for sustainable scheduling of delay-tolerant datacenter workloads.

REFERENCES

- [1] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.
- [2] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019.

- [3] C. Reiss, J. Wilkes, and J. L. Hellerstein, “Google cluster-usage traces: format + schema,” Google Inc., Technical Report, 2011.
- [4] L. A. Barroso and U. Hölzle, “The case for energy-proportional computing,” *Computer*, vol. 40, no. 12, pp. 33–37, 2007.
- [5] A. Radovanović *et al.*, “Carbon-aware computing for datacenters,” *IEEE Transactions on Power Systems*, 2021.
- [6] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, “CarbonScaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency,” in *Proc. ACM International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*, 2023.